

# **11-485/685/785, Fall 2026**

## **HW2P2: Bootcamp**

**TAs:** Mengchun Zhang, Bradley Warren, Ron Sarma, Euijin Hong, Boniface Godwin

## Reflection of HW1P2

- 1. What was challenging but you figured that out?**
- 2. Which strategies / resources helped you the most?**
- 3. What would you do differently if you could start over?**

# Reflection of HW1P2

1. What was challenging but you figured that out? → Great job
2. Which strategies / resources helped you the most? → Things to keep for HW2P2
3. What would you do differently if you could start over? → Things to change for HW2P2



# Overview of HW2P2

- HW2P2 is significantly more time consuming than HW1P2
- Models will be harder to develop, train, and converge
- Models must be written yourself and trained from scratch
- Please start early!
- Use what you learned from HW1P2 💪
- Strategize with your Study Team.

**Goal of this HW Bootcamp:**

**Help you to get started with HW2P2 🚀**

# Agenda

1. **Problem statement**
2. **Workflow**
  - a. Data loading and preprocessing
  - b. Building a model
  - c. Training, monitoring, testing, and Kaggle submission
  - d. Different loss functions for the task
3. **FAQs**

# Problem statement

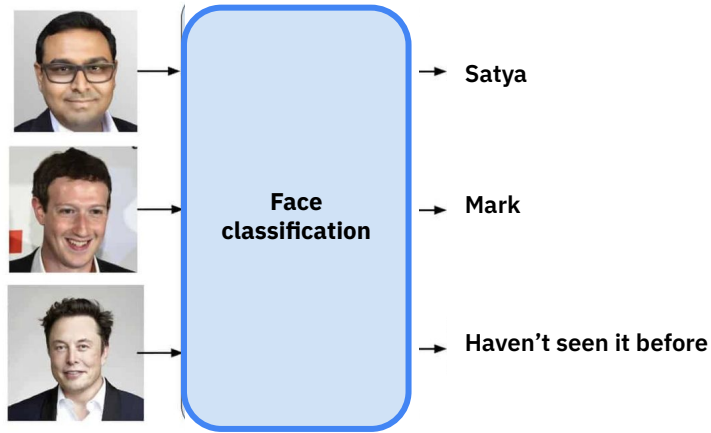
## HW2P2 objective

**Two** tasks, **one** model, **one** submission

- Face classification
- Face verification



# Face classification VS verification

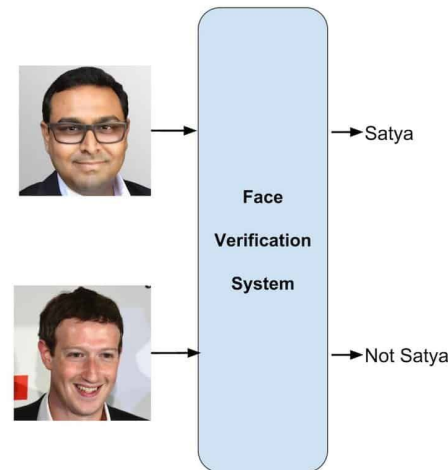
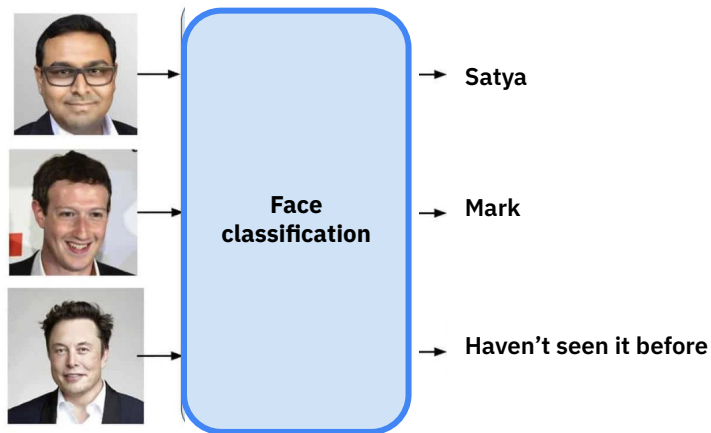


## Face classification (aka identification, recognition)

Given an image, who is this person? Person X, Y, or Z

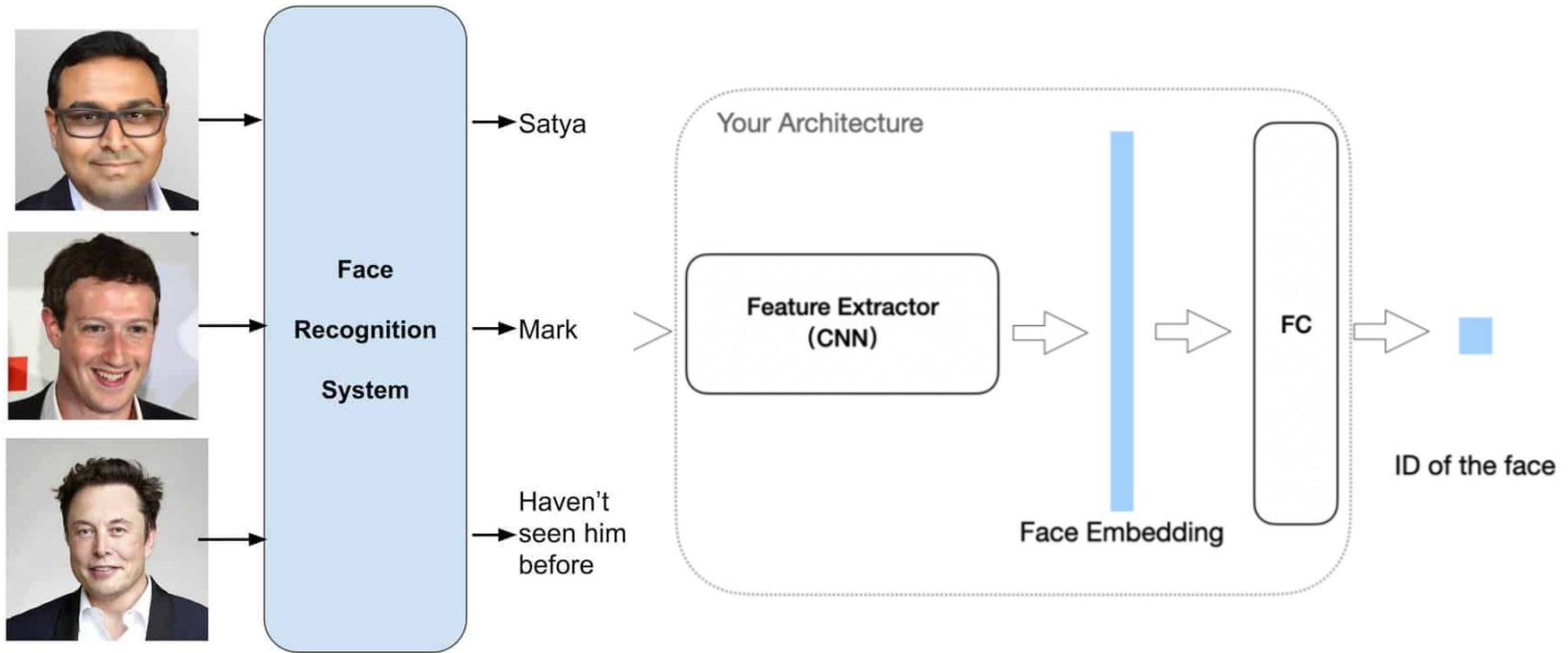
Example: can be used in applications like attendance system

# Face classification VS verification



| Face classification (aka identification, recognition)       | Face verification                                                         |
|-------------------------------------------------------------|---------------------------------------------------------------------------|
| Given an image, who is this person? Person X, Y, or Z       | Is the pair of Images given to you of the same person?                    |
| Example: can be used in applications like attendance system | Example: you can unlock your smartphone using your face, but others can't |

# How is this different from **Face Classification**?

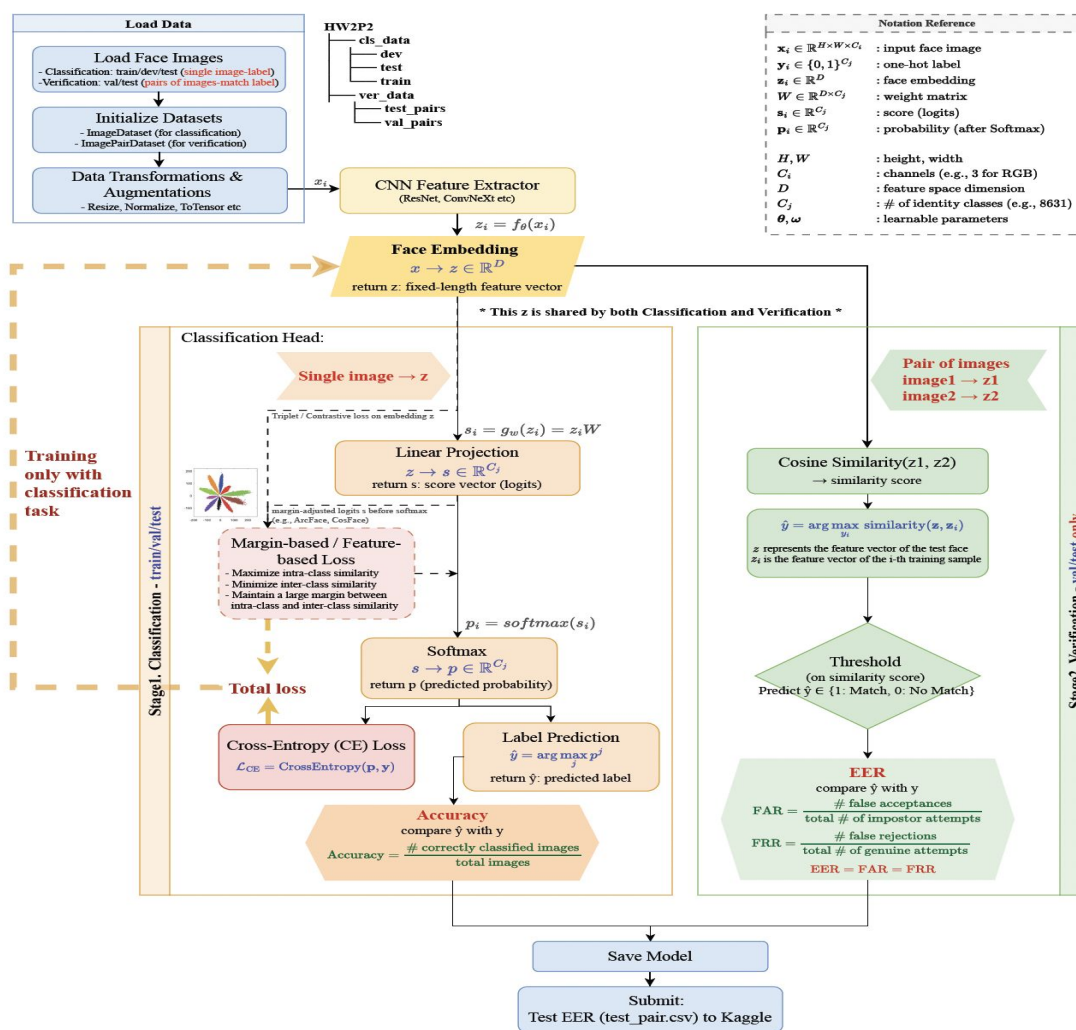


## Face Classification

- Closed set of predefined identities
- You **already know** all the possible people in your dataset
- Model **classifies an image into one of those known identities**

## Face Verification

- **Not limited to training identities**
- Goal: a generalized model
- **Verify whether any two faces belong to the same person** — even if unseen during training



# Workflow

# HW Parts 2 ideal workflow

**Step 0:** Download the notebook.

**Step 1:** Complete all #TODOs and ensure your code runs and reaches very low cutoff and submit the on Kaggle.

Important Update:

- No more Checkpoint Notebook
- Complete Homework Quiz on Canvas

**Step 2:** Divide the experiments among the study group members to achieve the high cutoff.



# HW Parts 2 components

**1:** Data loading and preprocessing

**2:** Building a model

**3:** Training and monitoring

**4:** Testing and Kaggle submission

**5:** Different loss functions and model fine-tuning

# **1. Data loading and Preprocessing**

# Dataset and Data Loader.

1. Classification Dataset Class
  - a. Train.
  - b. Validation.
  - c. Test.
2. Verification Dataset Class.
  - a. Validation.
  - b. Test.

For every dataset class, there is a data loader !

# Image Augmentations



# Why Do We Need Augmentations

- 1. Emulating More Data:** Transformations increase the perceived dataset size by applying various alterations to input images, leading to improved training and generalization.
- 2. Preventing Overfitting:** Transformations expose the model to 'new' versions of images in every epoch, reducing the memorization of specific images and promoting the learning of robust features
- 3. Invariance:** Training the model on transformed images teaches it to recognize objects independently of their orientation or position
- 4. Better Generalization:** Transformations diversify the training set, exposing the model to a wide variety of examples, which can enhance performance on unseen data

# Sample Augmentations

## Random Crop

Original image



## Random Rotation

Original image



# Random Horizontal Flip

Original image



**Good Transform**

# Random Vertical Flip

Original image



**Not So Good Transform**

# Sample Augmentations

## Color Jitter

Original image



## Random Perspective

Original image



# Transformation Tips

## Consider Normalising the Data:

- Use `torchvision.transforms.Normalize()` after calculating the mean and standard deviation of each pixel of each image of the dataset over all 3 channels(RGB)

## Common Issue:

`TypeError: Input tensor should be a torch tensor. Got <class 'PIL.Image.Image'>.`

- Please check the sequencing of your transforms. Read the documentation and verify the kind of input required.

## This Is What A Typical Transform Pipeline Could Look Like:

```
transforms = transforms.Compose([
    transforms.Random____(112),
    transforms.Random____(),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])
```

### Torchvision Transforms Illustrations URL:

[https://pytorch.org/vision/0.11/auto\\_examples/plot\\_transforms.html#sphx-glr-auto-examples-plot-transforms-py](https://pytorch.org/vision/0.11/auto_examples/plot_transforms.html#sphx-glr-auto-examples-plot-transforms-py)

# Final Images After Transformation

(After applying augmentations)

Training Samples



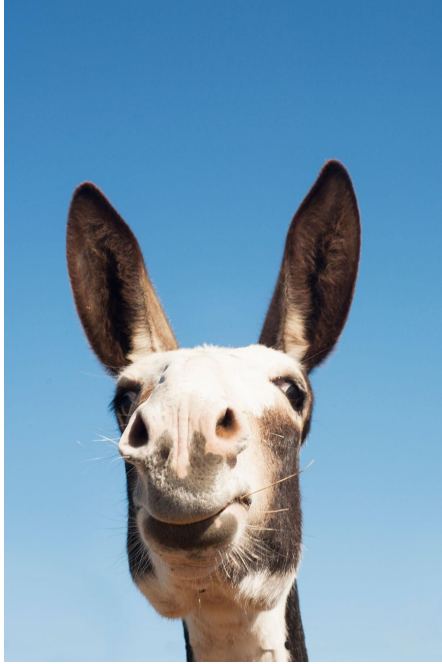
Validation Samples

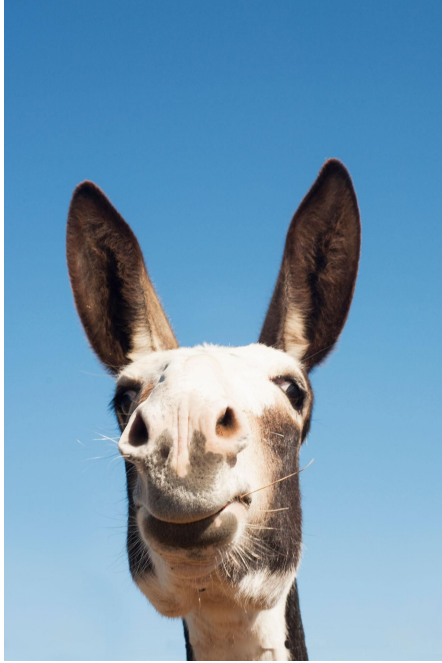


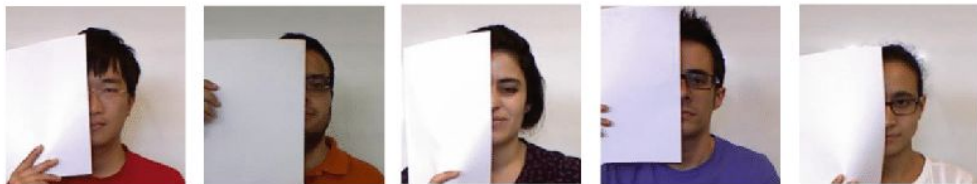
Test Samples







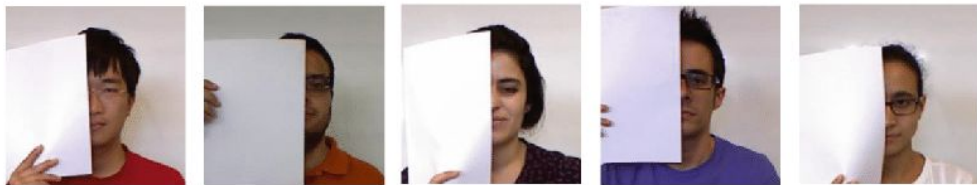




(a)



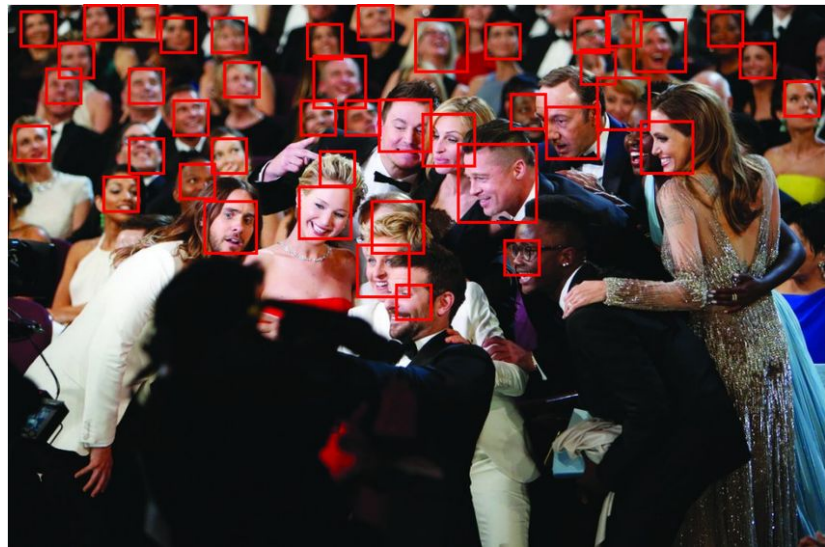
(b)



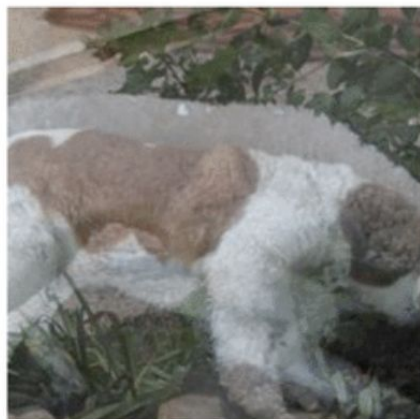
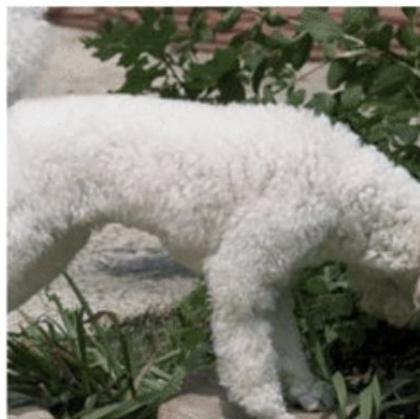
(a)



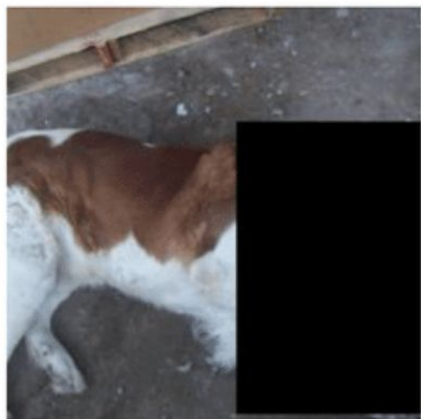
(b)



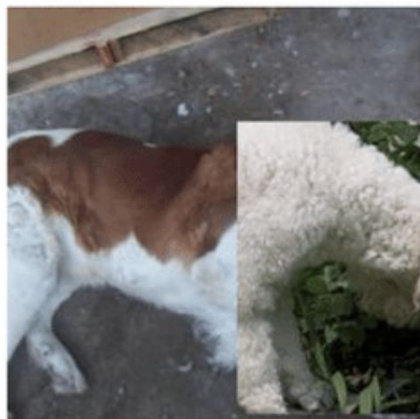
Original  
samples



Mixup



Cutout



Cutmix

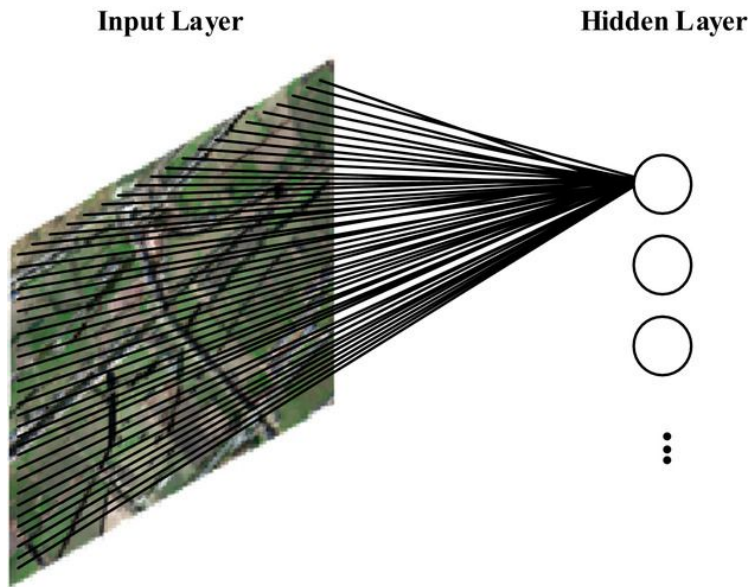
WE MIGHT HAVE A PROBLEM IF IT LOOKS  
LIKE THIS!!



## **2. Building a model**

# WHY NOT USE LINEAR LAYERS???

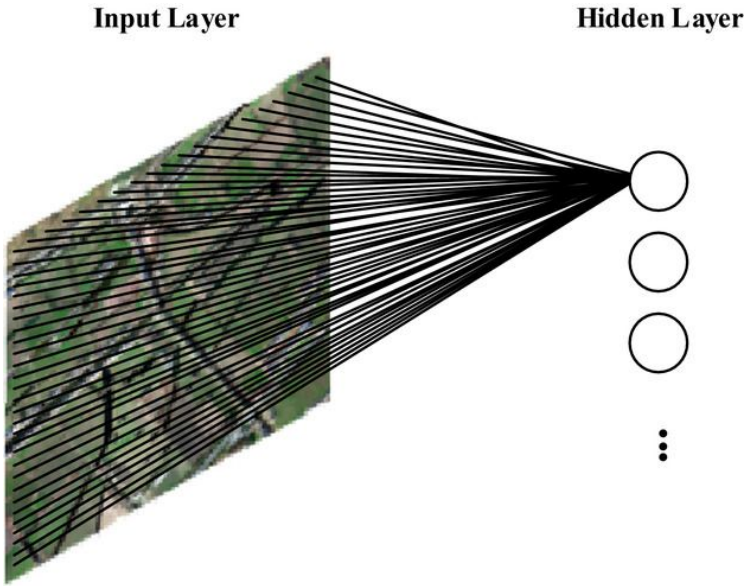




HINT: For a 1000 x 1000 image  
With 1 M Hidden Neurons.

HINT: For a 1000 x 1000 image  
(Grayscale)  
With 1 M Hidden Neurons.

That is  $\sim 10^{12}$  Trainable Parameters



Rather use something else...

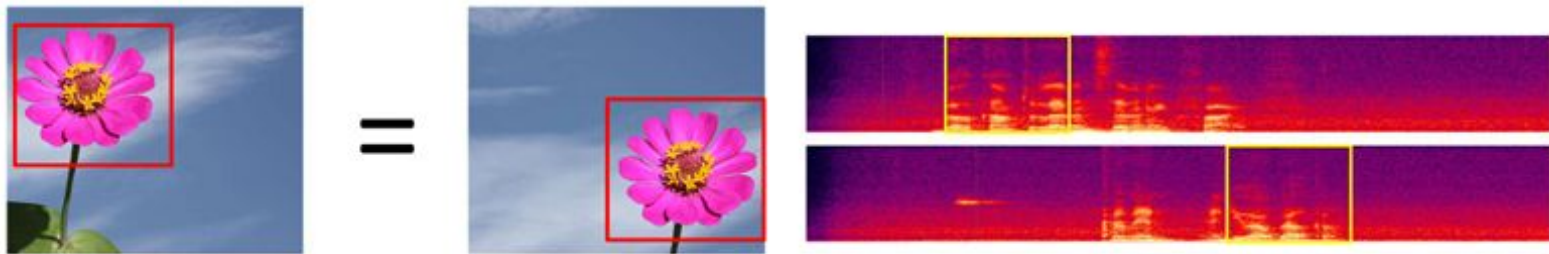


We Also Lose Spatial Features

# WHY NOT USE LINEAR LAYERS???

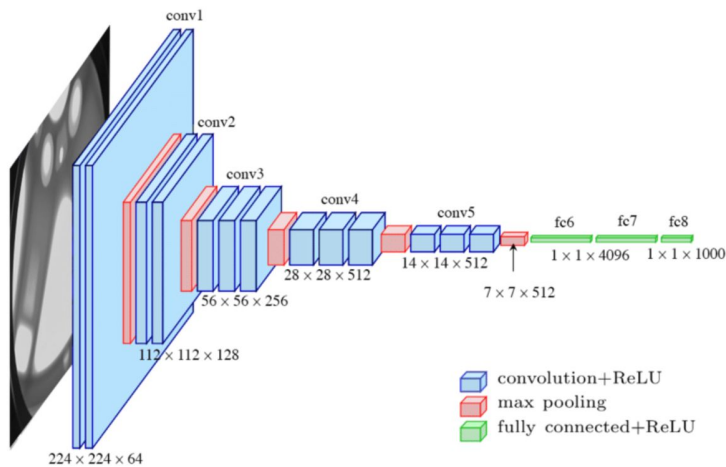
MLP sensitive to location of the image

The need for *shift invariance*



Shared filters

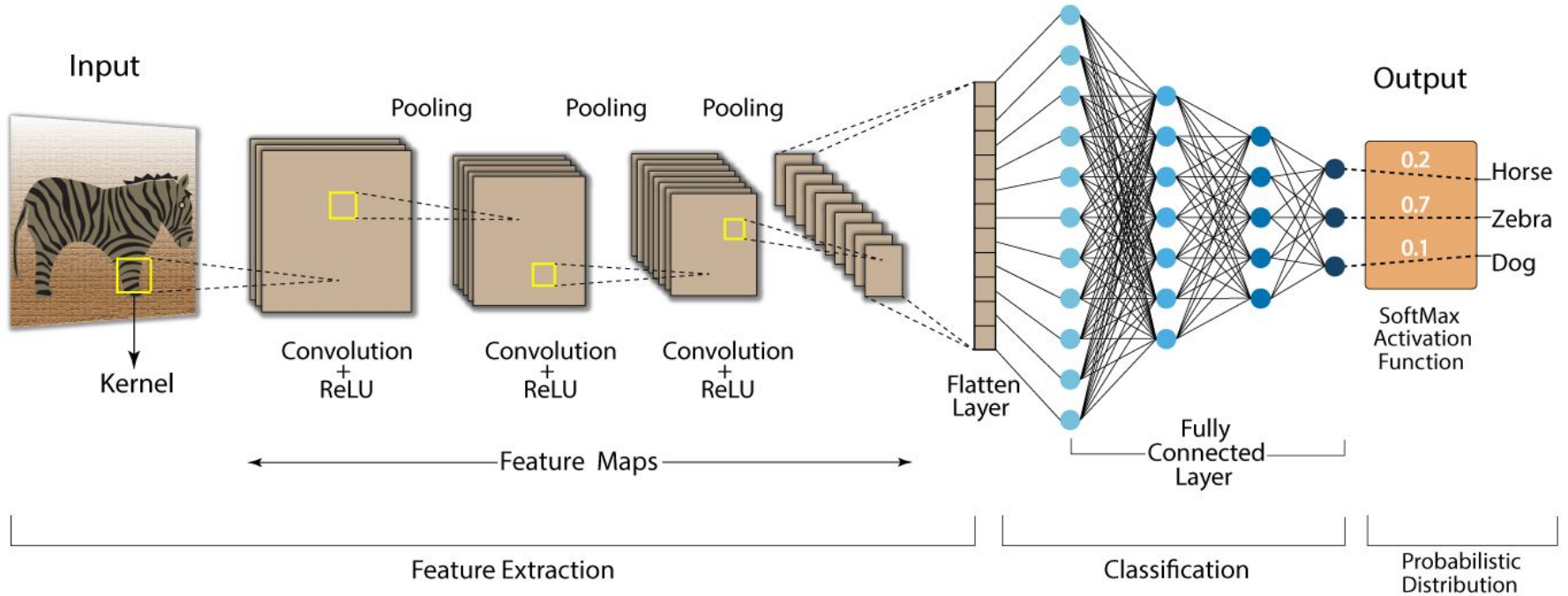
# OUR HERO: CNNs



BUT WHY?????



# Convolution Neural Network (CNN)

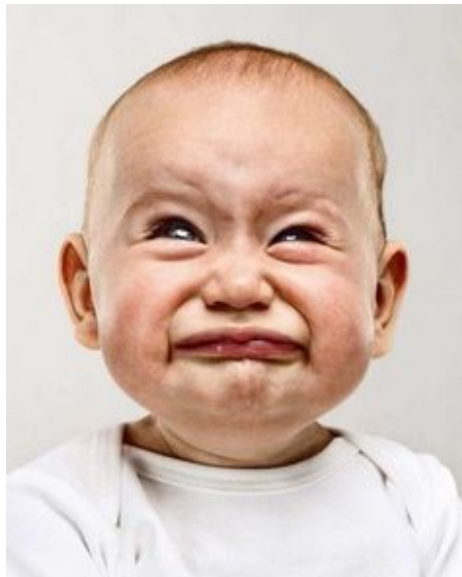
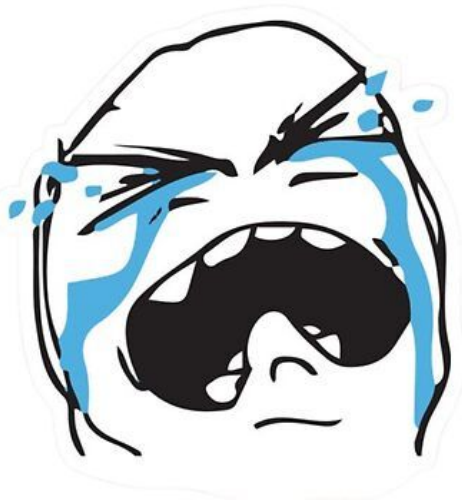


# CNN

1. They are good at capturing local spatial patterns (1D,2D data)
2. Good at feature detection.
3. They share parameters across local region, reducing number of parameters
4. Translational Invariance
5. Computational & data efficiency (pooling, inductive bias)



# ARE CNN'S ENOUGH ?



Smile

**NEED MORE COMPLEX  
ARCHITECTURES WITH  
DIFFERENT WAYS TO  
CONVERGE THE MODEL AND  
MITIGATE OVERFITTING**

# GOOD MODELS

(\*for this homework)

1. RESNET ([here](#))
2. SE-RESNET ([here](#))
3. ConvNext([here](#))
4. (ANY MAGICAL ARCHITECTURE)

# LET'S TALK ABOUT RESNETS



# Deep Residual Learning for Image Recognition

Kaiming He      Xiangyu Zhang      Shaoqing Ren      Jian Sun

Microsoft Research

{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

# ResNet

- Convert Paper -> Code [understand the blocks + main aspects]
- ResNet Blocks: [Basic Blocks and Bottleneck Blocks]
- ResNet Main Aspect: Residual Connections

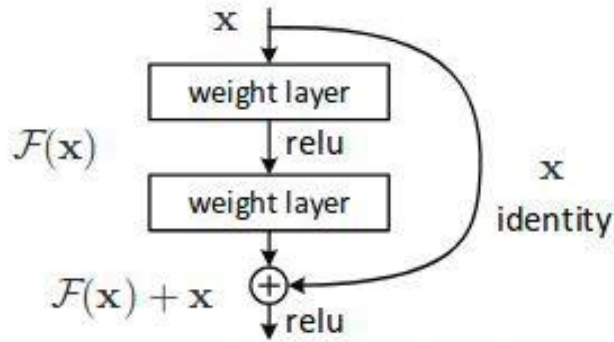


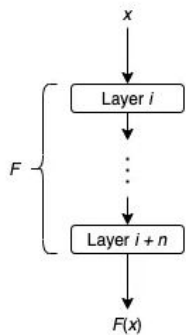
Figure 2. Residual learning: a building block.



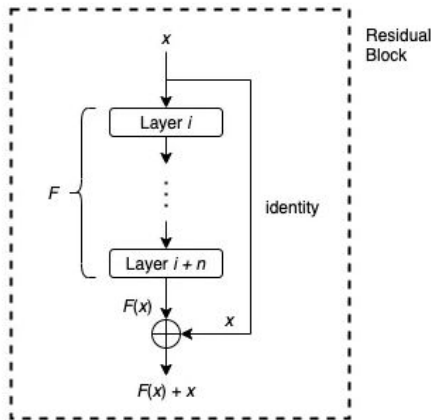
# Residual Connections

In traditional feedforward neural networks, data flows through each layer **sequentially**: The output of a layer is the input for the next layer.

**Residual connection** provides another path for earlier representations to reach latter parts of the neural network by **skipping** some layers.



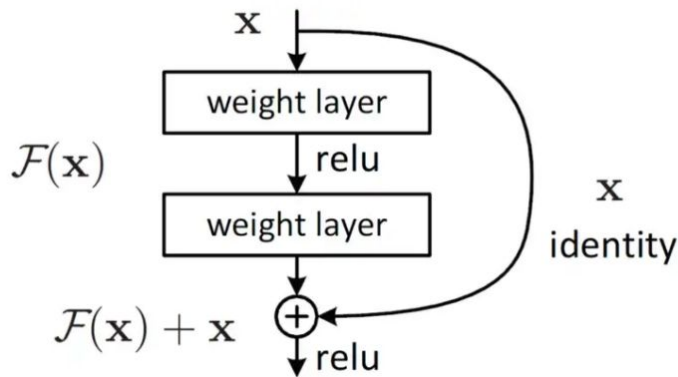
Traditional Feedforward  
without Residual Connection



With Residual Connection

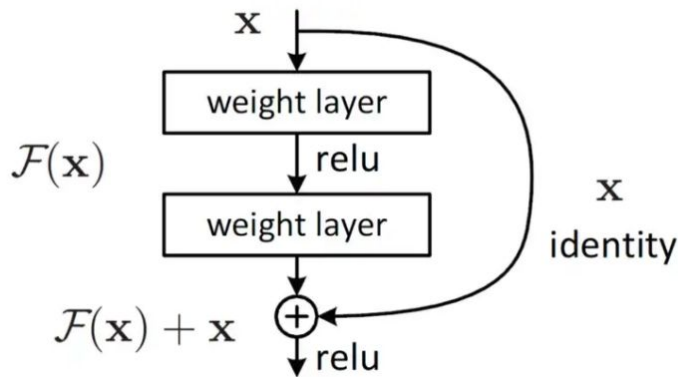
# Residual Connections

- The residual connection preserves  $x$  by identity mapping
- Then it performs element-wise addition  $F(x) + x$ .
- The whole architecture that takes an input  $x$  and produces output  $F(x) + x$  is usually called a residual block.
- Quite often, a residual block will also include an activation function such as ReLU applied to  $F(x) + x$ .

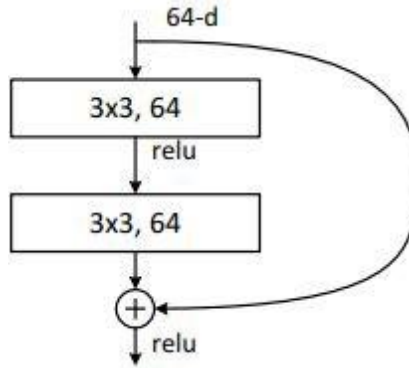


# How do they help??

- For feedforward neural networks, training a deep network is usually very difficult, due to problems such as **exploding gradients and vanishing gradients**.
- Neural networks with residual connections converge much more easily, even if the network has several hundreds layers.
- Preserves previous representations, block learns the residual, skip path of gradients
- Without residual connections, the gradients get unstable/weaker as they flow backward to these layers.

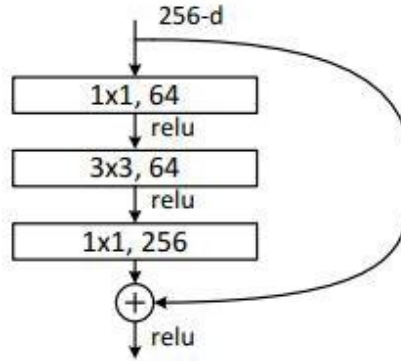


# ResNet: BasicBlock



- It's usually just a regular 3x3 convolution (then BN, ReLU), another 3x3 convolution (then BN).
- Then, a skip connection adding input and output, then ReLU.

# ResNet: Bottleneck Block



- A 256-channel input goes through a point-wise convolution, reducing channels to 64.
- Then, a  $3 \times 3$  regular convolution maintains channels at 64.
- Then, a point-wise convolution expands channels back to 256.
- Finally, the residual connection.

# Residual Connection - Basic Block

**\*\*Note: this is not the actual code, this is just an example :)\*\***

```
class BasicBlock(torch.nn.Module):
```

```
    def __init__(self, n_h):
```

```
        self.linear0 = torch.nn.Linear(n_h, n_h)
```

```
        self.linear1 = torch.nn.Linear(n_h, n_h)
```

```
        self.bn0 = torch.nn.BatchNorm1d(n_h)
```

```
        self.bn1 = torch.nn.BatchNorm1d(n_h)
```

```
        self.relu = torch.nn.ReLU(inplace=True)
```

```
    def forward(self, A0):
```

```
        R0 = A0
```

```
        Z0 = self.linear0(A0)
```

```
        BZ0 = self.bn0(Z0)
```

```
        A1 = self.relu(BZ0)
```

```
        Z1 = self.linear1(A1)
```

```
        BZ1 = self.bn1(Z1)
```

```
        A2 = self.relu(BZ1 + R0)
```

```
        return A2
```

# Residual Connection - Bottleneck Block

**\*\*Note: this is not the actual code, this is just an example :)\*\***

```
class Bottleneck(torch.nn.Module):
```

```
    def __init__(self, n_h):
```

```
        self.residual = torch.nn.Linear(n_h, n_h*4)
```

```
        self.linear0 = torch.nn.Linear(n_h, n_h )
```

```
        self.linear1 = torch.nn.Linear(n_h, n_h )
```

```
        self.linear2 = torch.nn.Linear(n_h, n_h*4)
```

```
        self.bn0 = torch.nn.BatchNorm1d(n_h )
```

```
        self.bn1 = torch.nn.BatchNorm1d(n_h )
```

```
        self.bn2 = torch.nn.BatchNorm1d(n_h*4)
```

```
        self.relu = torch.nn.ReLU(inplace=True)
```

```
    def forward(self, A0):
```

```
        R0 = self.residual(A0)
```

```
        Z0 = self.linear0(A0)
```

```
        BZ0 = self.bn0(Z0)
```

```
        A1 = self.relu(BZ0)
```

```
        Z1 = self.linear1(A1)
```

```
        BZ1 = self.bn1(Z1)
```

```
        A2 = self.relu(BZ1)
```

```
        Z2 = self.linear2(A2)
```

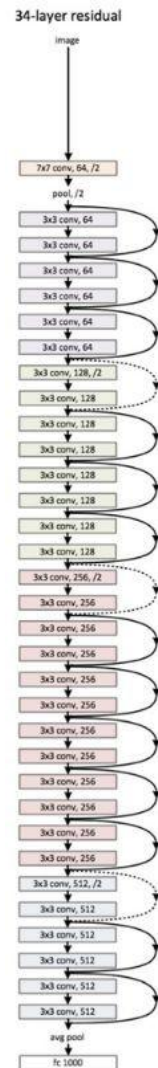
```
        BZ2 = self.bn2(Z2)
```

```
        A3 = self.relu(BZ2 + R0)
```

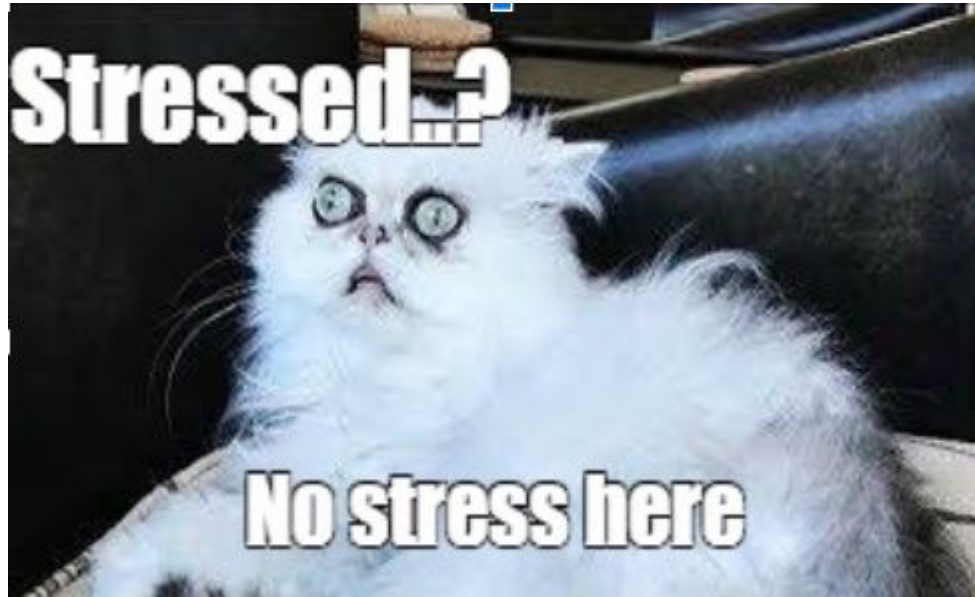
```
        return A3
```

# ResNet: Overall Architecture

| Layer Name | Output Size      | 18-Layer                                                                        | 34-Layer                                                                        | 50-Layer                                                                                              |
|------------|------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| cov1       | $112 \times 112$ | $7 \times 7, 64, \text{stride } 2$                                              |                                                                                 |                                                                                                       |
|            |                  | $3 \times 3 \text{ max pool, stride } 2$                                        |                                                                                 |                                                                                                       |
| cov2_x     | $56 \times 56$   | $\begin{bmatrix} 3 \times 3, & 64 \\ 3 \times 3, & 64 \end{bmatrix} \times 2$   | $\begin{bmatrix} 3 \times 3, & 64 \\ 3 \times 3, & 64 \end{bmatrix} \times 3$   | $\begin{bmatrix} 1 \times 1, & 64 \\ 3 \times 3, & 64 \\ 1 \times 1, & 256 \end{bmatrix} \times 3$    |
| cov3_x     | $28 \times 28$   | $\begin{bmatrix} 3 \times 3, & 128 \\ 3 \times 3, & 128 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3 \times 3, & 128 \\ 3 \times 3, & 128 \end{bmatrix} \times 4$ | $\begin{bmatrix} 1 \times 1, & 128 \\ 3 \times 3, & 128 \\ 1 \times 1, & 512 \end{bmatrix} \times 4$  |
| cov4_x     | $14 \times 14$   | $\begin{bmatrix} 3 \times 3, & 256 \\ 3 \times 3, & 256 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3 \times 3, & 256 \\ 3 \times 3, & 256 \end{bmatrix} \times 6$ | $\begin{bmatrix} 1 \times 1, & 256 \\ 3 \times 3, & 256 \\ 1 \times 1, & 1028 \end{bmatrix} \times 6$ |
| cov5_x     | $7 \times 7$     | $\begin{bmatrix} 3 \times 3, & 512 \\ 3 \times 3, & 512 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3 \times 3, & 512 \\ 3 \times 3, & 512 \end{bmatrix} \times 3$ | $\begin{bmatrix} 1 \times 1, & 512 \\ 3 \times 3, & 512 \\ 1 \times 1, & 2048 \end{bmatrix} \times 3$ |
|            | $1 \times 1$     | average pool, 1000-d fc, softmax                                                |                                                                                 |                                                                                                       |
| FLOPs      |                  | $1.8 \times 10^9$                                                               | $3.6 \times 10^9$                                                               | $3.8 \times 10^9$                                                                                     |



# AHHHHHHHHHH



# BREATHE!!!!

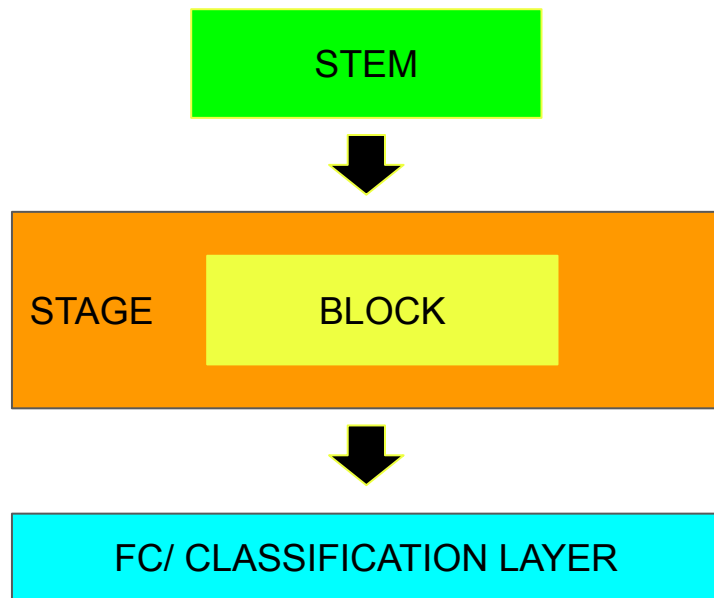


# General Architecture Flow

- CNN architectures are divided into stages, which are divided into blocks.
  - Each “stage” consists of (almost) equivalent “blocks”
  - Each “block” consists of a few CNN layers, BN, and ReLUs.
- To understand an architecture, we mostly need to understand its blocks.
- All that changes for blocks in different stages is the base # of channels

# General Architecture Flow

- However, you do need to piece these blocks together into a final model.
- The general flow is like this:
  - Stem
  - Stage 1
  - Stage 2
  - ...
  - Stage n
  - Classification Layer



# Summary

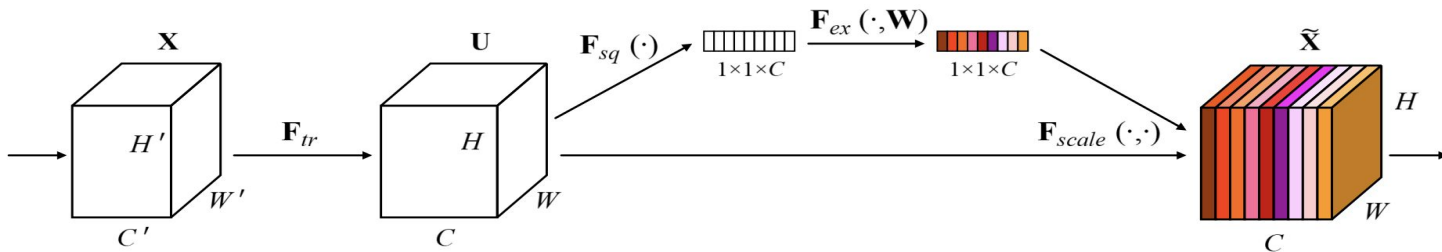
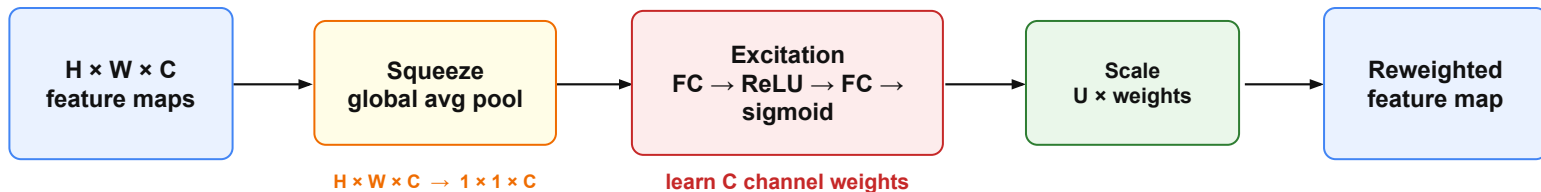
- **A convolution mixes information from both different channels and different spatial locations (pixels)**
- **Convert Paper -> Code [understand the blocks + main aspects]**
- **Residual connection provides another path for earlier representations to reach latter parts of the neural network by skipping some layers.**
- **Remember parameter limit for this HW is 30 million.**

# **Other Model Architectures**

# Squeeze-and-Excitation block

A small channel-recalibration module that can be inserted into many CNN backbones.

Input feature map U



## Core innovation

The block uses global context to decide which channels should be emphasized or suppressed for this specific input image.

## Strength

It is cheap and modular: the main convolutional backbone stays almost the same, but every block gets an input-dependent channel gate.

## Limitation

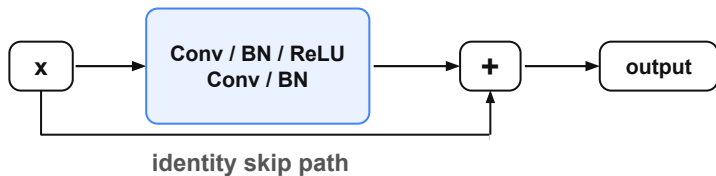
It recalibrates channels; it does not create a new spatial structure by itself. If the base network is weak, SE will not fix everything.

**Think of SE as a learned per-image volume knob for channels, not as a replacement for the CNN block.**

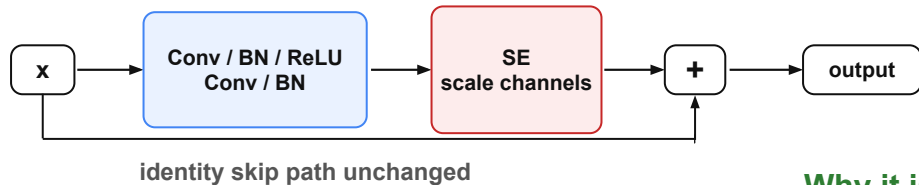
# SE-ResNet: ResNet block + channel recalibration

The skip connection stays. The SE module is applied to the residual branch before adding the identity path.

## Plain residual block



## SE-ResNet block

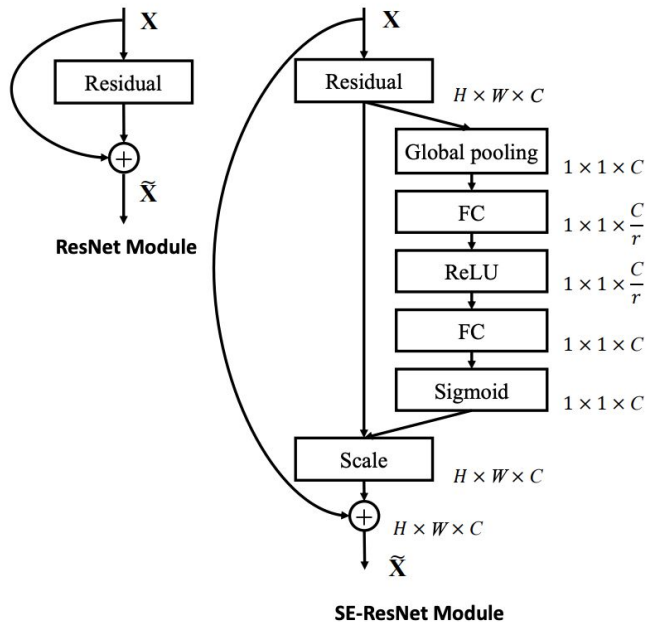


## Why it is attractive

It gives a familiar ResNet backbone a stronger channel-selection mechanism with small overhead.

## What to watch

More capacity can overfit, and channel attention does not remove the need for good augmentation, normalization, and checkpoint selection.

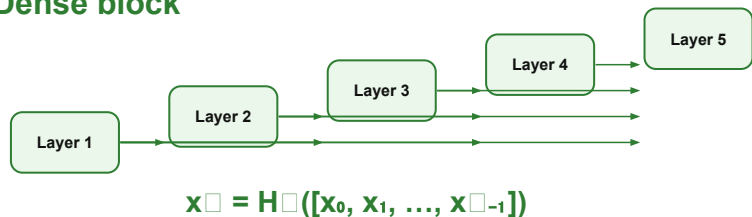


**SE-ResNet changes the block minimally but adds a clear new inductive bias.**

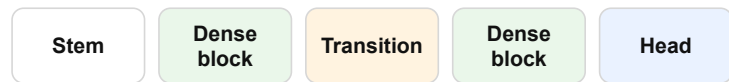
## DenseNet: keep earlier features available

DenseNet changes the connection pattern: later layers receive all previous feature maps by concatenation.

## Dense block



## Typical network



Transition layers reduce resolution and control channel growth between dense blocks.

## Innovation

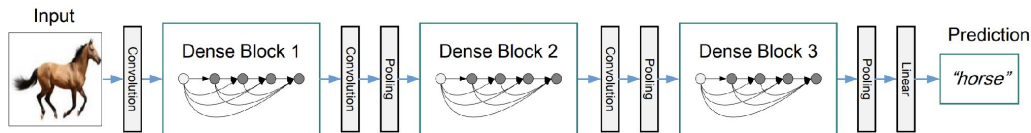
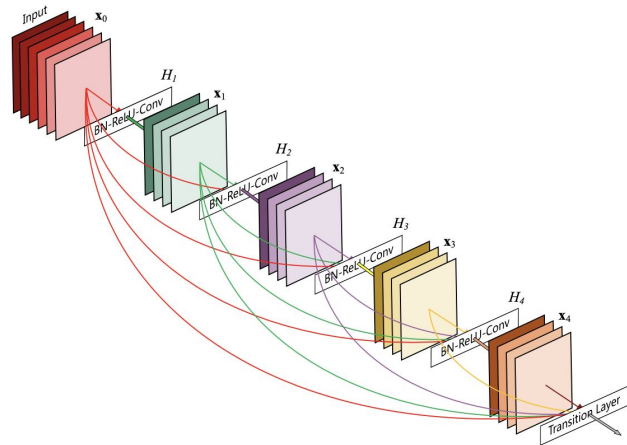
Instead of adding a shortcut, DenseNet concatenates old and new features. This preserves information explicitly.

## Good fit

Strong feature reuse can help build discriminative embeddings, especially when training very deep plain networks would be unstable.

## Tradeoff

Concatenation grows channels. The model can be slower and heavier in memory than it looks from parameter count alone.

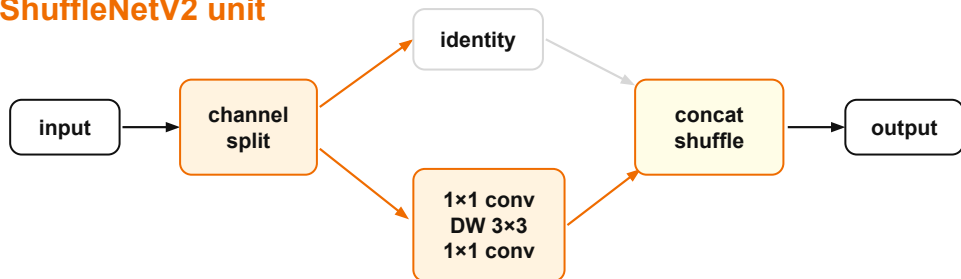


**DenseNet is useful when you want stronger feature reuse, but it is less convenient for fast iteration than lightweight families.**

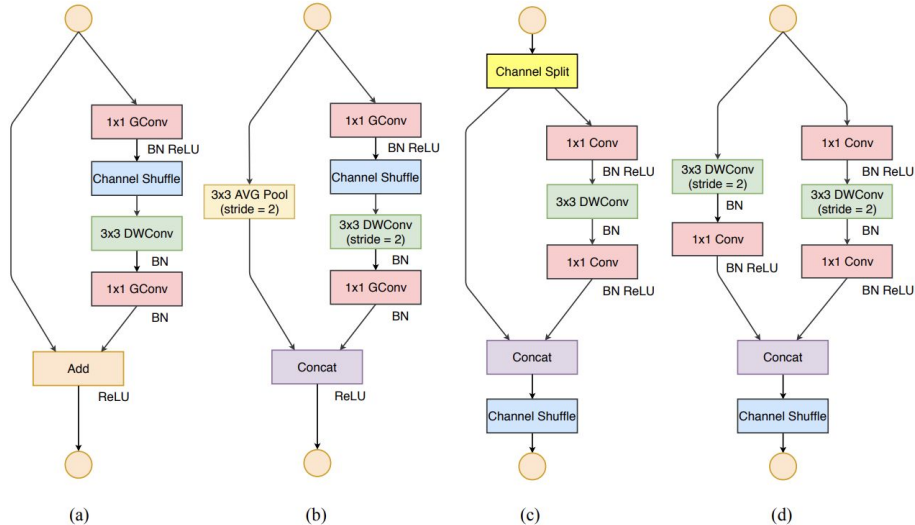
# ShuffleNetV2: efficiency designed around real speed

The key idea is not simply fewer FLOPs; the unit is designed to reduce memory access and fragmentation.

## ShuffleNetV2 unit



Only part of the channels are transformed; shuffle restores communication across channels.



## Innovation

A practical block design: channel split, lightweight transform branch, concatenate, then channel shuffle.

## Good fit

Fast to train and cheap enough for many ablations. A wider 2.0× version can still have enough capacity for this task.

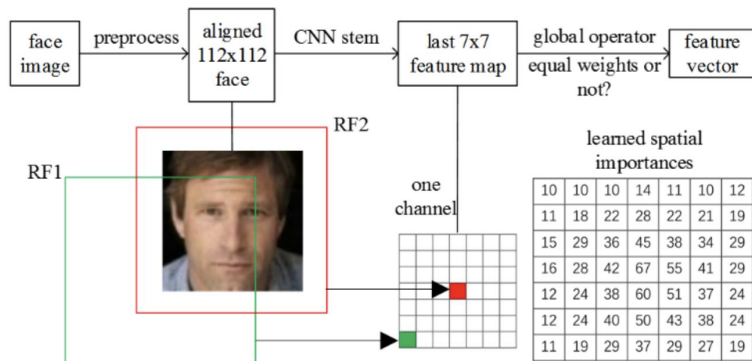
## Tradeoff

Because the block is optimized for efficiency, performance depends heavily on width, training setup, and loss weighting.

**ShuffleNetV2 is not merely a lightweight baseline; its efficient block design can still provide strong representation capacity.**

# MobileFaceNet: compact CNN for face embeddings

This family was designed for real-time face verification, not generic ImageNet classification.



## Backbone flow



The face-specific detail is the end of the network: global depthwise convolution replaces plain global average pooling.

## Innovation

Use mobile inverted-bottleneck style blocks, then a global depthwise convolution tailored to face-verification embeddings.

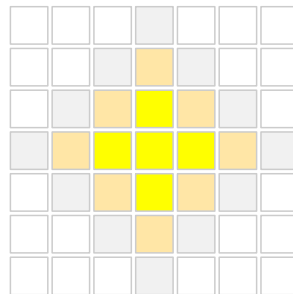
## Good fit

Good for fast experiments, low memory use, and testing whether a face-specific compact design is enough.

## Tradeoff

Small capacity can limit classification accuracy. It may need careful width, embedding dimension, and loss balancing.

## Why GDConv is used

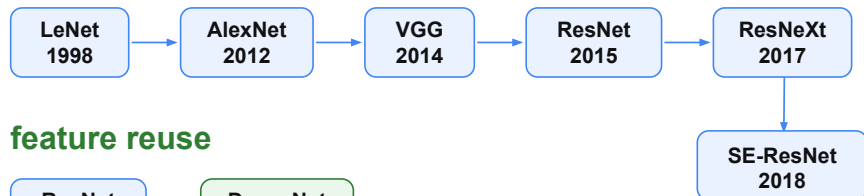


learned spatial weighting for the final face feature map

**Designed specifically for face verification, MobileFaceNet prioritizes compact discriminative embeddings and efficiency, rather than maximizing classification capacity.**

# Evolution of ImageNet CNN Backbones

## classic depth / residual route



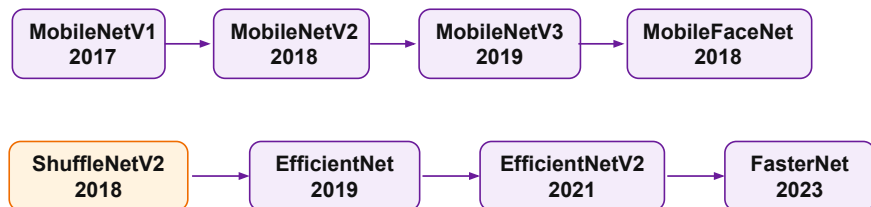
## feature reuse



## multi-branch CNN modules



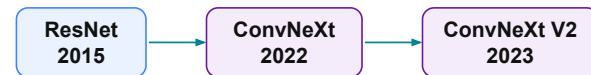
## mobile / efficient CNN route



## re-parameterized CNN route

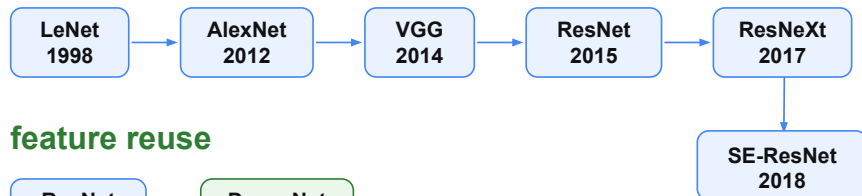


## modern pure ConvNet route



# Evolution of ImageNet CNN Backbones

## classic depth / residual route



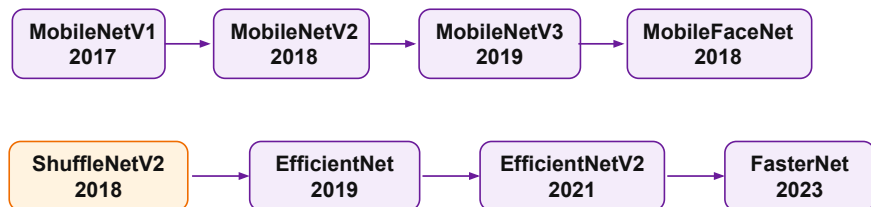
## feature reuse



## multi-branch CNN modules



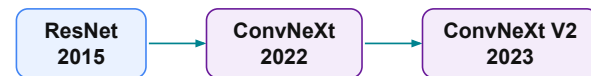
## mobile / efficient CNN route



## re-parameterized CNN route



## modern pure ConvNet route



How should information flow?

How should channels interact?

How should spatial information be processed?

How can we reuse features?

How do we reduce computation without destroying representation quality?

How should we scale the network when we have more compute?

# Summary

- A normal convolution mixes information from **both different channels and different spatial locations (pixels)**
- A depth-wise convolution only mixes information **over spatial locations**
  - Different channels do not interact.
- A point-wise convolution only mixes information **over different channels**
  - Different spatial locations do not interact
  - **Remember parameter limit for this HW is 30 million.**

# ConvNeXt

- This is a very new paper, a state-of-the-art architecture.
- However, its **intuitions are very similar to MobileNetV2**.
- Again, remember that to understand a paper, we just really need to understand its **blocks**.
- Just a single block type for ConvNeXt
- **Read the paper** for details on stages/channel sizes, etc.
  - We recommend ConvNeXt-T size which has less than 35M parameters.
  - A ConvNeXt block typically looks like this:
    - Depth-wise convolution with a large kernel, often  $7 \times 7$
    - LayerNorm instead of BatchNorm
    - Point-wise MLP implemented with  $1 \times 1$  convolutions
    - GELU activation
    - Residual connection

### **3. Training, monitoring, and testing**

# Binary Classifier & Why Accuracy Can Be Misleading

## Binary Classifier

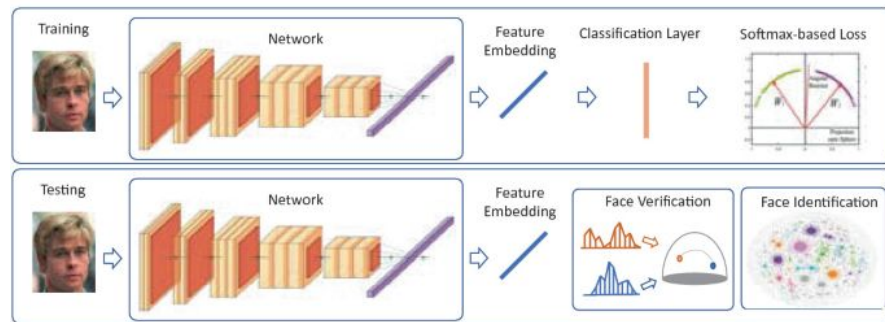
- A model that predicts one of two classes
  - Examples: **Spam / Not Spam**, **Fraud / Legit**, **Accept / Reject**
- Outputs:
  - A **label** (0 or 1), often based on a **score + threshold**

## Confusion Matrix Basics

- **True Positive (TP)**: Correctly predicts positive
- **True Negative (TN)**: Correctly predicts negative
- **False Positive (FP)**: Predicts positive, but actually negative
- **False Negative (FN)**: Predicts negative, but actually positive

## Problem with Using Accuracy Alone

- $\text{Accuracy} = (\text{TP} + \text{TN}) / \text{Total}$
- Can be **misleading with imbalanced data**
  - Example: 99% non-fraud  $\rightarrow$  predicting “non-fraud” always gives 99% accuracy 😞
- Accuracy ignores *types* of errors:
  - Some mistakes are worse than others



# Error Rates That Actually Matter

## False Positive Rate (FPR)

- “How often do we falsely raise an alarm?”
- Formula:  $FPR = FP / (FP + TN)$
- Example:
  - Legit users flagged as fraud
  - Good emails marked as spam

## False Negative Rate (FNR)

- “How often do we miss a real positive?”
- Formula:  $FNR = FN / (FN + TP)$
- Example:
  - Fraud that slips through
  - Disease not detected

## Key Tradeoff

- Lowering FPR usually **increases** FNR
- Controlled by the **decision threshold**

## Confusion Matrix

|                        | Actually Positive (1) | Actually Negative (0) |
|------------------------|-----------------------|-----------------------|
| Predicted Positive (1) | True Positives (TPs)  | False Positives (FPs) |
| Predicted Negative (0) | False Negatives (FNs) | True Negatives (TNs)  |

# Equal Error Rate (EER)

## What is EER?

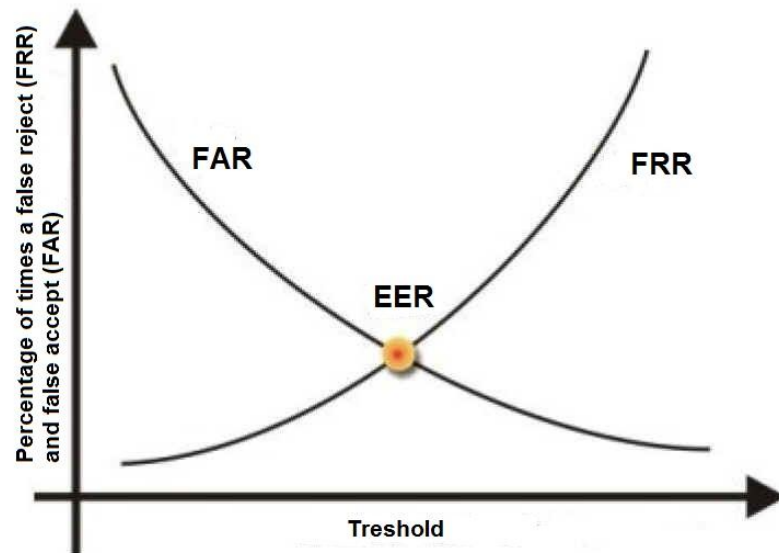
- Point where:  $FPR = FNR$
- Occurs at a **specific threshold**

## Intuition

- Treats false positives and false negatives as **equally costly**
- Answers the question:  
“At what operating point are both types of errors balanced?”

## Why EER is Useful

- Common in:
  - Biometrics (face, voice, fingerprint verification)
  - Security & authentication systems
- Gives a **single-number summary** of classifier performance
  - Lower EER = better model



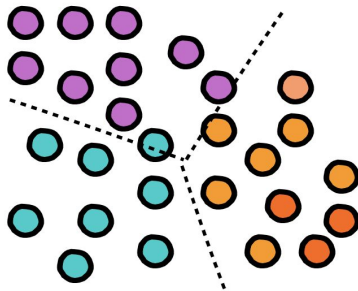
# Training Tips

**Goal:** Learn *useful representations* efficiently and stably.

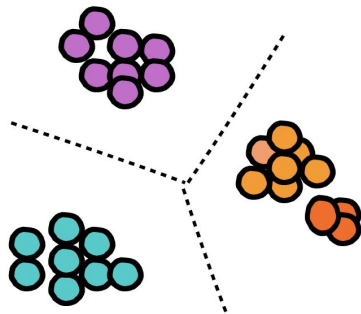
- **Start simple, then scale**
  - Verify your pipeline works with a *small subset* (e.g., 1–5 epochs, small batch).
  - Debug losses, shapes, and labels **before full training**.
- **Loss & objective alignment**
  - Classification loss helps learn separable features.
  - Verification loss (contrastive / metric-based) shapes embedding space.
  - Make sure you understand *what each loss is optimizing*.
- **Optimization sanity**
  - Adam / AdamW is a strong default.
  - If loss explodes → lower LR.
- **Reproducibility matters**
  - Fix seeds.
  - Log configs (LR, batch size, augmentations).
  - Small changes can shift Kaggle scores a lot.

## Common Pitfalls

- Training “looks fine” but embeddings are useless.
- Over-augmenting early and killing signal.
- Forgetting `.train()` / `.eval()` mode switches.



Separable Features (e.g. classification )



Discriminative Features (e.g. metric learning )

# Monitoring: Trust the Right Metrics

**Goal:** Know *when* your model is actually improving.

## What to Track:

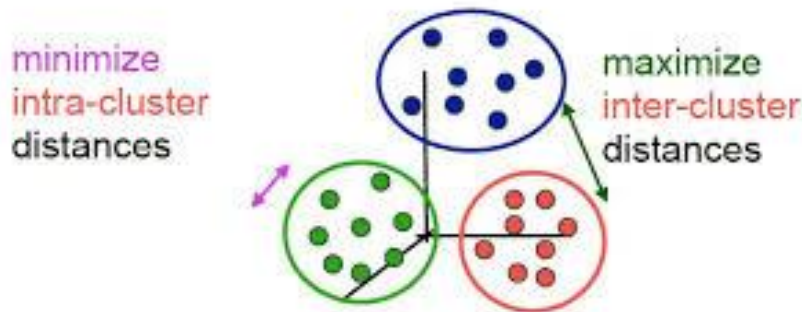
- **Training vs Validation loss (But CAREFULLY!)**
  - Training ↓, Validation ↑ → overfitting.
  - Both flat → underfitting or LR issue.
- **Task-specific metrics**
  - Classification accuracy ≠ good verification performance.
  - Always monitor **verification metric** (EER, cosine similarity trends).
- **Embedding quality**
  - Monitor mean intra-class vs inter-class distance.
  - If they overlap → embeddings are not separating identities.

## Practical Advice

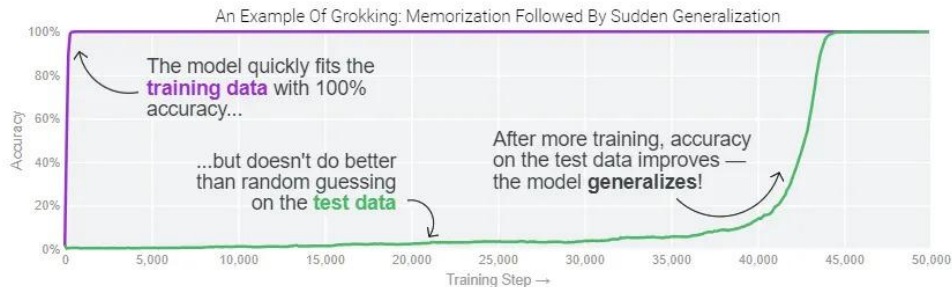
- Plot metrics **every epoch** (not just at the end).
- Save **best model by validation metric**, not last epoch.
- Watch for *validation noise* — small bumps are normal.

## Common Pitfalls

- Picking checkpoints based on training loss.
- Ignoring validation split distribution mismatch.
- Over-optimizing a single metric blindly.



# Rethinking Overfitting in Modern Deep Learning



- Classic intuition:
  - Large train–val gap  $\Rightarrow$  stop training (overfitting)
- Modern reality:
  - Model can still improve **after training accuracy reaches 100%**
- Delayed generalization exists:
  - Models may suddenly generalize after long plateaus
  - Known as **Grokking / Delayed Generalization**
- Practical takeaway:
  - Train **until validation metric stops improving**
  - Or use a **fixed LR schedule** (e.g., CosineAnnealingLR for 50 epochs) and let it run
- Don't panic over:
  - Early plateaus
  - Temporary train–val gaps

# How to Tackle Overfitting (CNNs)

There are many ways to improve generalization — don't try everything at once

- **Start with high-impact, low-risk tricks** (recommended first):
  - **Label smoothing** (often a *huge* boost)
  - **Stochastic Depth**
  - **DropBlock**
  - **Dropout before the final classification layer**
- If needed, try additional techniques:
  - Data augmentation tweaks
  - Optimizer / LR schedule changes
  - Weight decay tuning
- Good reference:
  - “Bag of Tricks for Image Classification with CNNs”
  - <https://arxiv.org/abs/1812.01187>

| (pre-)training config       | ConvNeXt-T/S/B/L<br>ImageNet-1K<br>224 <sup>2</sup> |
|-----------------------------|-----------------------------------------------------|
| optimizer                   | AdamW                                               |
| base learning rate          | 4e-3                                                |
| weight decay                | 0.05                                                |
| optimizer momentum          | $\beta_1, \beta_2=0.9, 0.999$                       |
| batch size                  | 4096                                                |
| training epochs             | 300                                                 |
| learning rate schedule      | cosine decay                                        |
| warmup epochs               | 20                                                  |
| warmup schedule             | linear                                              |
| layer-wise lr decay [6, 10] | None                                                |
| randaugment [12]            | (9, 0.5)                                            |
| label smoothing [65]        | 0.1                                                 |
| mixup [85]                  | 0.8                                                 |
| cutmix [84]                 | 1.0                                                 |
| stochastic depth [34]       | 0.1/0.4/0.5/0.5                                     |
| layer scale [69]            | 1e-6                                                |
| gradient clip               | None                                                |
| exp. mov. avg. (EMA) [48]   | 0.9999                                              |

## **4. Different loss functions for the task**

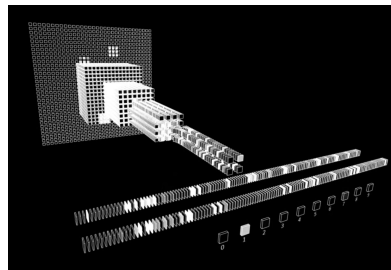
# Face Classification vs. Verification

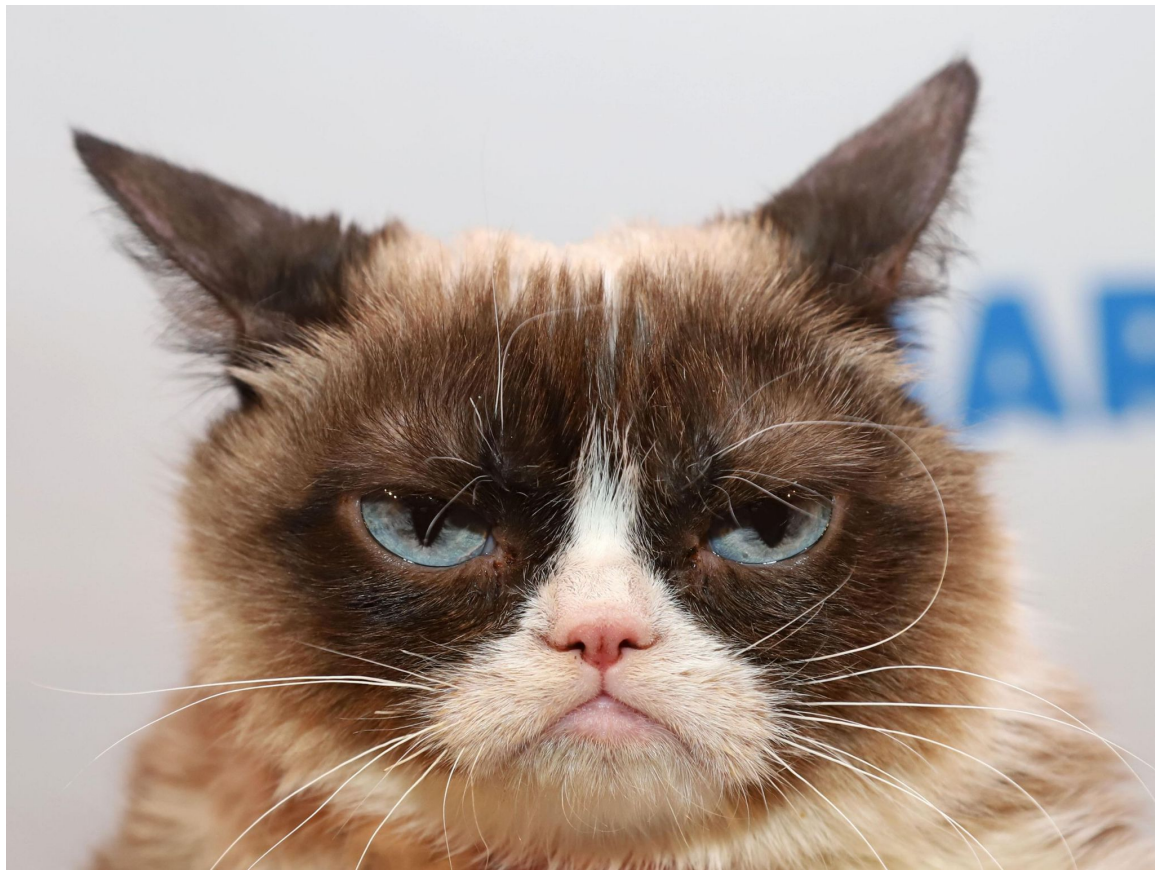
Goal of Face **classification**: *who is this?*

Goal of Face **verification**: *are they the same person?*

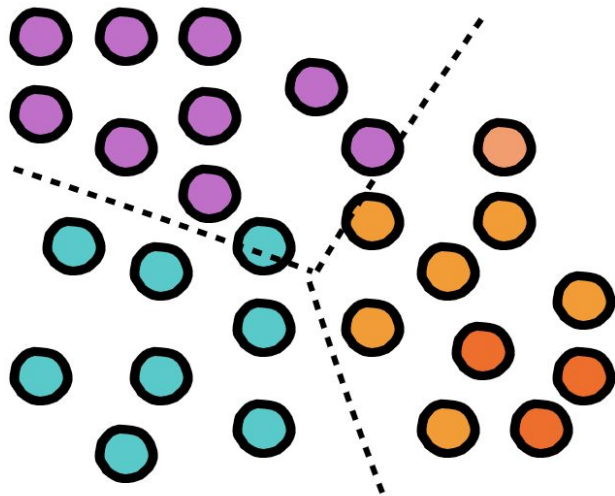
We want to learn a **similarity metric**

*A notion of distance, trickier in high dimensions*

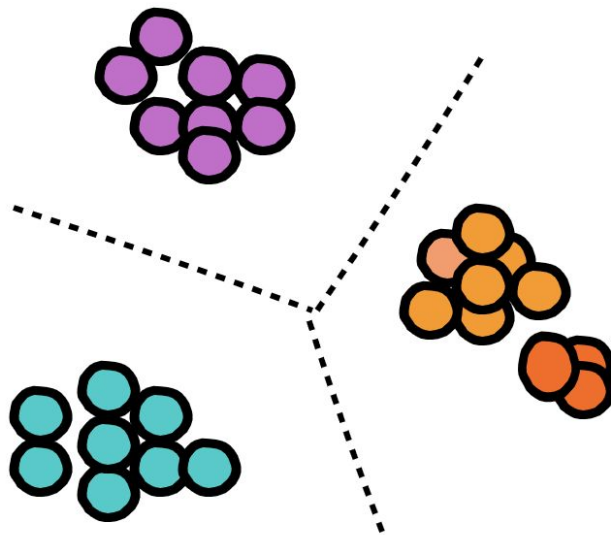




# Face Classification vs. Verification



Separable Features (e.g. classification )



Discriminative Features (e.g. metric learning )

# Cross Entropy Loss

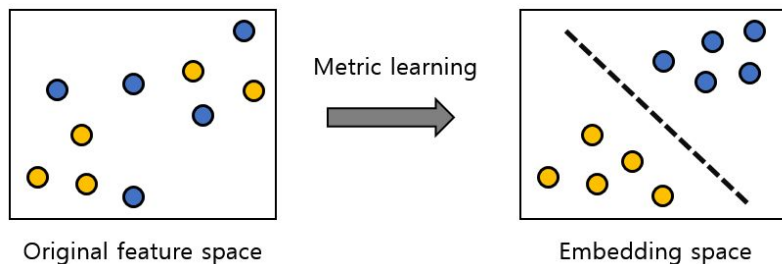
$$l_{\text{CE}} = \sum_j^C -y^j \log p^j \approx \max_{k \in [C], k \neq y} (s^k) - s^y \quad s^j = \mathbf{z} \mathbf{w}_j^T = \|\mathbf{z}\| \|\mathbf{w}_j^T\| \cos \theta_j$$

- CE is a classification loss, not explicitly designed for verification
- Features learned with CE are often not sufficiently **discriminative** for verification tasks
- CE typically requires a **large number of classes** to learn well-separated embeddings

**Challenge: Two samples can be correctly classified but still be poorly arranged in feature space.**

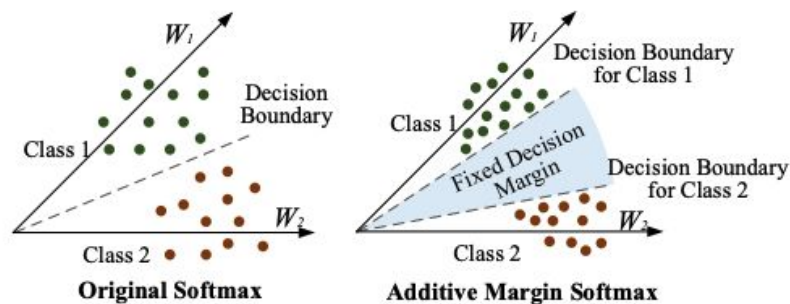
# Two Loss Paradigms

## Metric/Pairwise



- ❑ [Centre Loss](#)
- ❑ Triplet Loss
- ❑ Circle Loss
- ❑ N-Pair Loss
- ❑ Contrastive Loss...

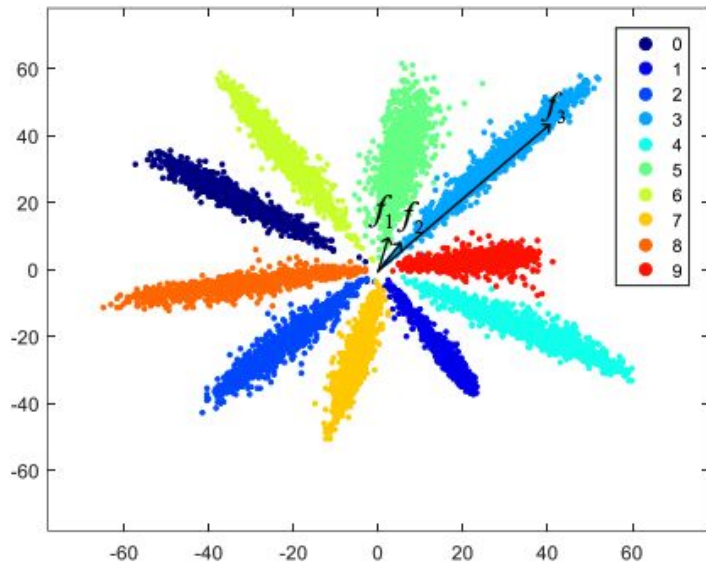
## Margin-based Softmax



- ❑ AM-Softmax
- ❑ [SphereFace](#)
- ❑ CosFace
- ❑ [ArcFace](#)
- ❑ CombinedMarginFace

# Feature Visualization of Cross-Entropy

The radial starburst pattern

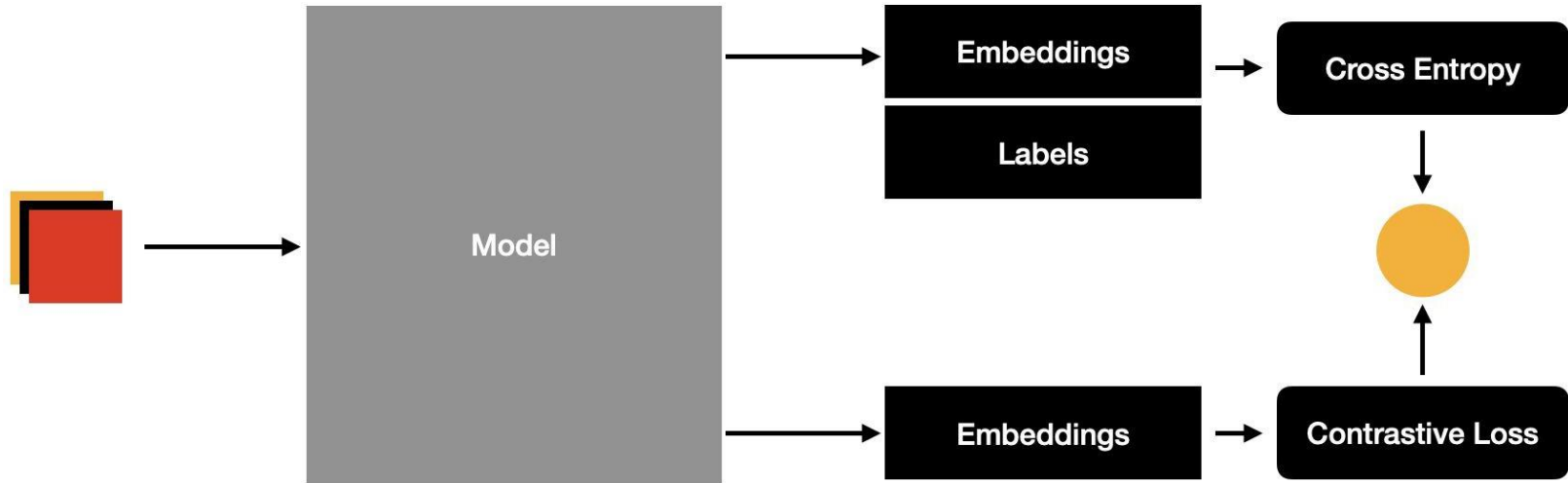


**Larger** intra-class similarity  
(Smaller intra-class distance)

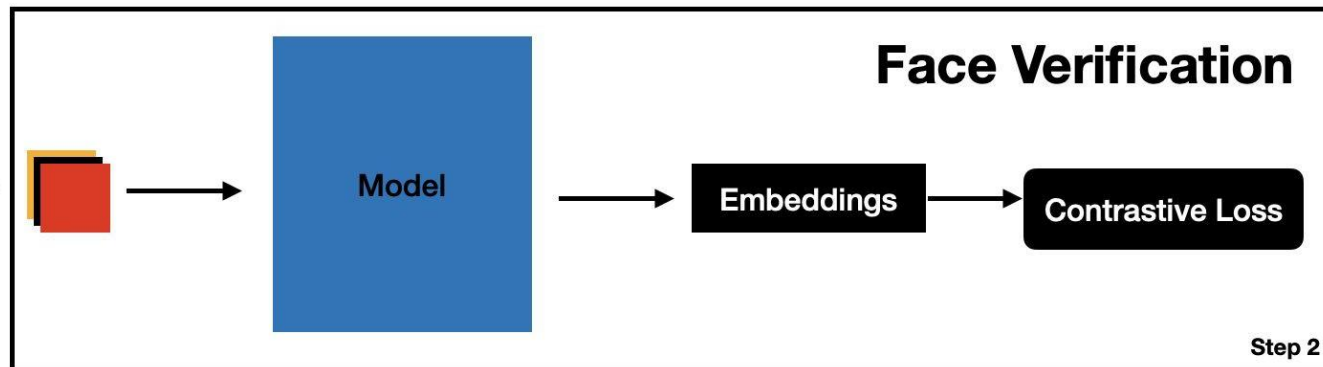
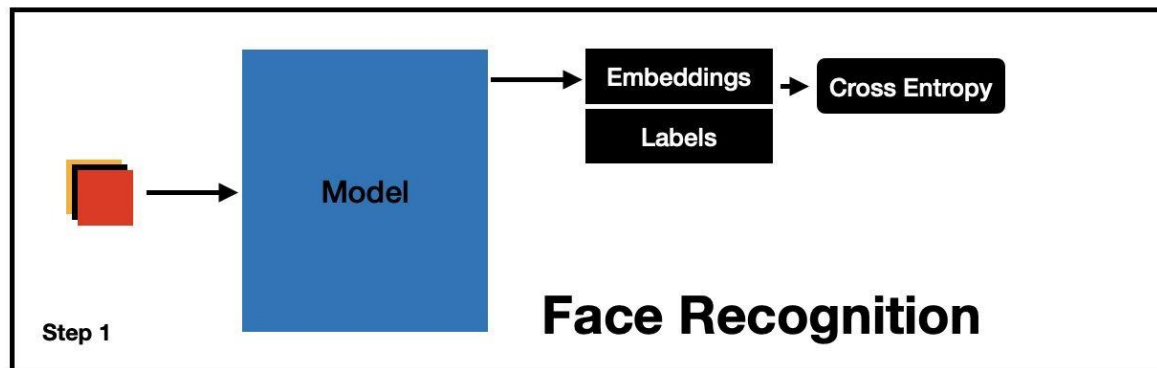
**Smaller** inter-class similarity  
(Larger inter-class distance)

The geometry of the embedding space matters more  
than the class logits.

# Approach 1 - Joint Loss Optimization



# Approach 2 - Sequential (Fine-tuning)



# How to Get Started

- Use a simple network and cross-entropy loss for early deadline
- Try better architectures with cross-entropy loss
- Try data augmentation
- Try other loss functions
- Fine-tune your cross-entropy loss with other loss functions

Recommended Watch: [Rec 0.20](#) and [Rec 0.21](#)

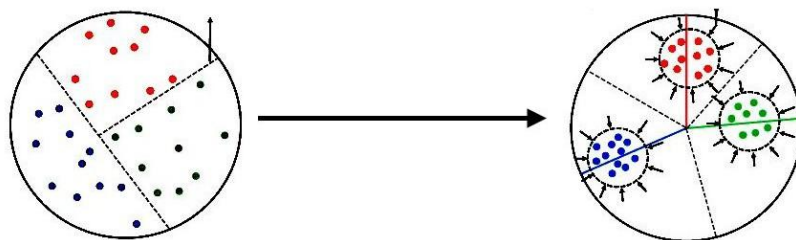
PyTorch Metric Learning

<https://kevinmusgrave.github.io/pytorch-metric-learning/losses/>

# Centre Loss

$$\mathcal{L}_C = \frac{1}{2} \sum_{i=1}^m \|\mathbf{x}_i - \mathbf{c}_{y_i}\|_2^2$$

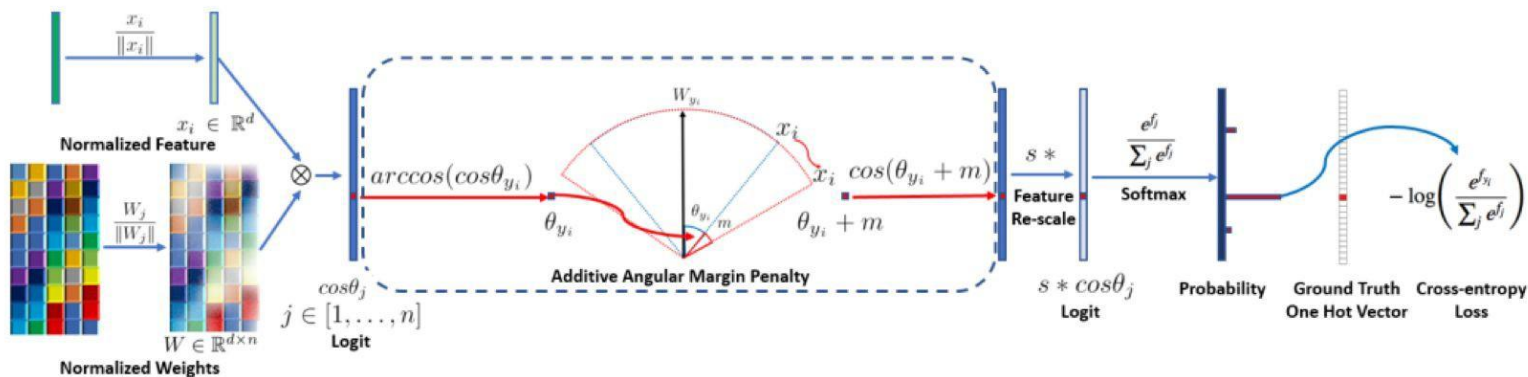
- **Increases the disparity between classes using softmax**
- **Increases inter-class distance by reducing intra-class Euclidean distance by assigning centers to each class.**
- **Calculating the centre for each class, is difficult**



# ArcFace

$$L = -\frac{1}{N} \sum_{i=1}^N \ln \frac{\exp \left\{ s \cdot \cos(\theta_{y_i, i} + m) \right\}}{\exp \left\{ s \cdot \cos(\theta_{y_i, i} + m) \right\} + \sum_{j \neq y_i} \exp \left\{ s \cdot \cos(\theta_{j, i}) \right\}}$$

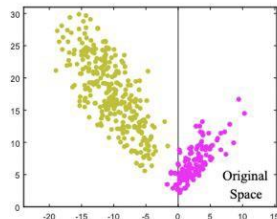
- Builds on the concepts of the sphere face and cos face.
- Replaced the multiplicative angular margin in CosFace, with an additive margin 'm'
- The additive factor of 'm' has found to lead to better convergence as compared to its multiplicative counterpart in Sphere Face.



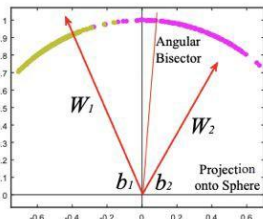
# Sphere Face

$$L_{\text{ang}} = - \sum_i \ln \frac{\exp \left\{ \|\mathbf{x}_i\| \cos(m \cdot \theta_{y_i, i}) \right\}}{\exp \left\{ \|\mathbf{x}_i\| \cos(m \cdot \theta_{y_i, i}) \right\} + \sum_{j \neq y_i} \exp \left\{ \|\mathbf{x}_i\| \cos(\theta_{j, i}) \right\}}$$

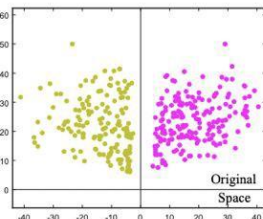
- **Makes use of an angular margin, imposed by  $\theta$**
- **The learned features construct a discriminative angular distance equivalent to the geodesic distance on a hypersphere manifold**
- **$\theta$  :- denotes the type of decision boundary learned, which leads to different margins for different classes**



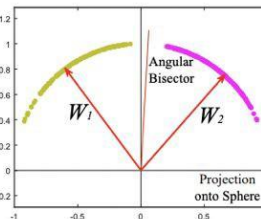
(a) Original Softmax Loss



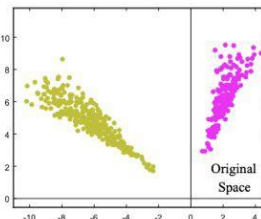
(b) Original Softmax Loss



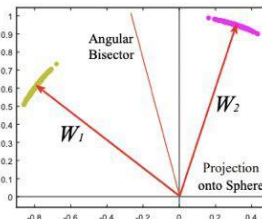
(c) Modified Softmax Loss



(d) Modified Softmax Loss



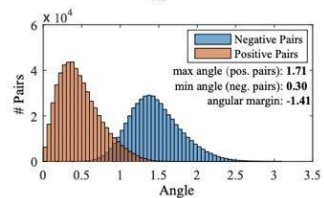
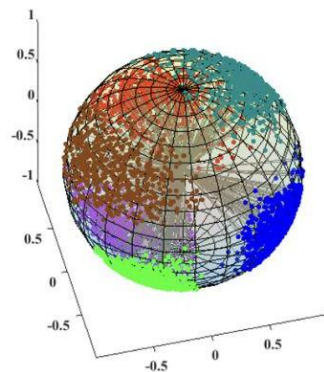
(e) A-Softmax Loss



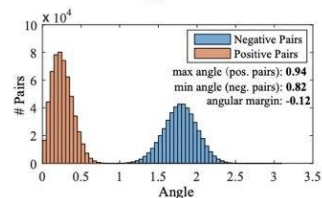
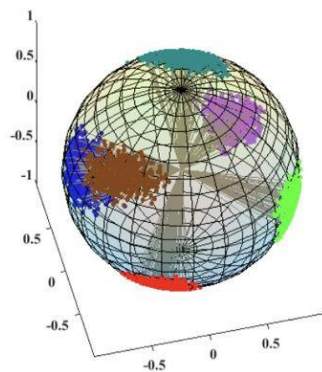
(f) A-Softmax Loss

# Sphere Face

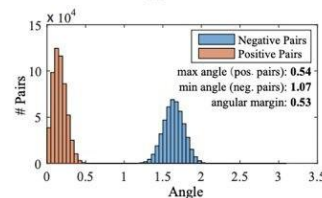
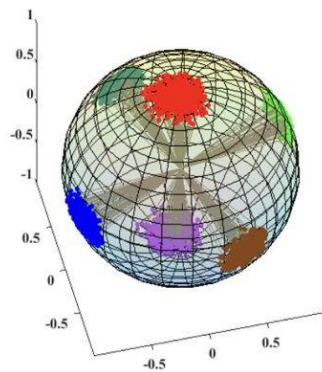
$$M \propto \text{Angular Margin}$$



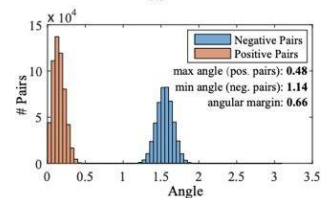
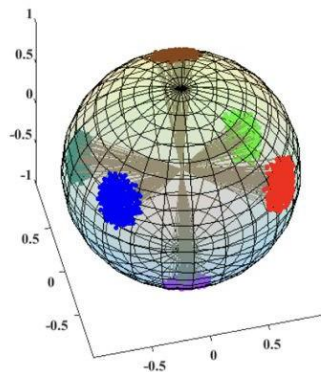
A-Softmax (m=1)



A-Softmax (m=2)



A-Softmax (m=3)



A-Softmax (m=4)

# FAQ

**Thanks!**

**See you on Piazza and in OHs!**